

Service Discovery using Volunteer Nodes for Pervasive Environments

M J Kim, M Kumar and B A Shirazi
University of Texas at Arlington
Box 19015, Arlington TX 76019
{mkim,kumar,shirazi}@cse.uta.edu

Abstract.

We propose a service discovery architecture called VSD (Service Discovery based on Volunteers) for heterogeneous and dynamic pervasive computing environments. The proposed architecture uses a small subset of the nodes called volunteers that perform directory services. Relatively stable and capable nodes serve as volunteers, thus recognizing node heterogeneity in terms of mobility and capability. We discuss characteristics of VSD architecture and methods to improve connectivity among volunteers for higher discovery rate. By showing that VSD performs quite well compared to a broadcast based scheme in MANET scenarios, we validate that VSD is a flexible and adaptable architecture appropriate for dynamic pervasive computing environments. VSD incorporates several novel features: i) handles dynamism and supports self-reconfiguration; ii) provides physical locality and scalability; and iii) improves reliability and copes with uncertainty through redundancy by forming overlapped clusters.

Keywords: Service discovery; Pervasive computing; heterogeneity; uncertainty; locality; MANET

1. Introduction

Pervasive computing is an emerging and very challenging area of research, envisaging computing devices that are embedded in the environments and yet disappear into the background requiring minimum user attention [1]. Devices are invisible to users, but required to collaborate and expected to provide benefits to users. To achieve above goals in pervasive computing environments, devices need to join the network easily and make use of services dynamically. Thus there is a demand for flexible and practical service discovery/interaction architecture suitable for dynamic pervasive computing environments.

Let us consider two main characteristics of pervasive computing environments: *heterogeneity* and *uncertainty*. Heterogeneity applies to devices, network media and environments themselves. Every device has different h/w components, operating systems, capabilities and services. In addition, the amount/types of resources and network connectivity vary [2]. Some devices are abundant in resources of computation, communication and energy while others have scarce resources. In some environments, most nodes move dynamically but in others, only a few nodes are dynamic. For example, in an office, only handheld or wearable devices are moving but other devices such as a fax machine are almost static. However, in shopping malls, only a few devices including commercial displays, coffee machines and ATMs are static whereas devices such as PDAs and cellular phones are carried by shoppers. Uncertainty also applies to reliability and stability of specific servers/services or network connectivity [3]. For example, Internet connectivity or application specific cache service is not always available in some situations.

There are a number of existing/proposed protocols for service discovery [4][5][6][7]. Even though the existing protocols have different design goals and architectures, basically they operate in either a centralized or a decentralized manner. Protocols that operate in a centralized manner [4][5] need specialized servers to provide directory services. On the other hand, the latter type of protocols [5][6][7] use multicast for service providers and service requestors to advertise and request services respectively. Both types of protocols are unsuitable for pervasive computing environments because static servers and administration are not expected to be available semi-permanently and multicast is prone to poor scalability due to high overhead.

In this paper, we propose VSD (Service Discovery based on Volunteers), a service discovery architecture for heterogeneous and uncertain pervasive computing environments. In VSD, a small subset of the nodes called volunteers, perform directory services in the

system. Relatively stable (less mobile) and capable (resourceful) nodes serve as volunteers. Without loss of generality, in this paper, we assume multi-hop wireless ad hoc networks common in pervasive computing environments. The VSD architecture is not limited to any specific routing protocols or physical network media. VSD exploits node heterogeneity in terms of mobility and capability. In addition, VSD copes with uncertainty thanks to duplicate service information through overlapped clusters.

The paper is organized as follows. In Section 2, we discuss related work. Section 3 introduces VSD protocols and Section 4 investigates its characteristics. Comprehensive simulation studies are presented in Section 5 and we conclude and present future work in Section 6.

2. Related work

Service discovery architectures/protocols have been developed and investigated with an emphasis on auto-configurable (autonomous) networks. Sun's Jini [4], Service Location Protocol (SLP) of IETF [5] and Microsoft's Universal Plug and Play (UPnP) [6] are prominent among them. Jini operates with centralized directory servers and UPnP performs in a decentralized manner. SLP works in the two operation modes with and without centralized directory services. All the above protocols are not well suited for pervasive computing environments due to limitations in support for node mobility or scalability.

There have also been considerable researches for lookup services in peer to peer networks [8][9][10][11]. The lookup services usually concern scalability but require more effective ways to support physical locality on real topology. In addition, lookup systems that make use of distributed hash tables (DHTs), strictly assign IDs to resources and tightly control data placement within the network. Therefore, they are impractical and inflexible especially for pervasive computing environments characterized by heterogeneous services and resources.

There have been several proposals for service discovery in ad hoc networks recently [7][12][13][14]. Schemes in [7] and [13] achieve flexible service matching by employing a service hierarchy tree to describe and discover services. However, all nodes are required to maintain the hierarchy tree and matching mechanism is not simple. In [12], authors proposed the service discovery architecture supported by network layer. Besides the formation of a virtual backbone, the periodic service registration and management of multicast group incur high overhead. Ad hoc networks are common in pervasive computing. Thus the above service discovery proposals targeting ad hoc networks

may be more preferable choices than others such as Jini and SLP that operate in administered networks.

Clustering has gained attention in ad hoc and sensor network research areas because clustering can support various functionalities effectively such as QoS support, resource efficient routing and energy saving information fusion [15][16][17][18]. In [18], layered clusters are used for localized service propagation for service discovery in ad hoc networks.

3. VSD Architecture

3.1. The role of volunteers

The core idea of VSD is that some nodes become volunteers to provide directory services to other nodes in the network. In pervasive computing environments, nodes may differ from each other in terms of resource availability and degree of mobility. We consider three kinds of resources - communication, computation and energy. Degree of mobility is the relative value to indicate dynamism of a node and a function of average speed of the node¹. Even though the amount of resources and degree of mobility may change continuously, nodes may set them semi-permanently by considering average values over a given sample period of time to minimize update overheads. In addition, we assume that every node has different willingness value to be a volunteer set by its user or owner. A node that has abundant resources, a lower degree of mobility and a higher willingness value has a higher chance to be a volunteer. The volunteer with which a node registers is called a *local volunteer* of the node. The node that registers with its local volunteer is called a *client* of the local volunteer. Any node (a volunteer or a client) can be a service provider (SP) as well as a service requestor (SR). Each client has k local volunteers and the k local volunteers maintain same directory entry for the common client. k is a system parameter and we discuss proper choice of the value for k in the next section.

Volunteers maintain a list of neighbor volunteers and a service directory for its region². Volunteers initially set a number of maximum clients (n_{max_c}) to limit workload. Volunteers constitute logical overlay networks to cooperate for service discovery and proactive advertisement (explained in detail later).

Volunteers periodically announce their existence by announcement messages consisting of message *id*, source address, TTL_a , hop count and lifetime. TTL_a is

¹ If nodes have difficulty to measure the speed, they may set degree of mobility using other methods, e.g. by monitoring neighbors or facilitating routing/MAC protocol.

² The region of a volunteer is a set of nodes including its clients and the volunteer itself. We use a cluster as the same meaning of a region, thus they are exchangeable in this paper.

time to live (TTL) field to limit boundary of the announcement. Hop count field is used to figure out the hop distance from the volunteer at each client. Whenever a client receives an announcement message, it caches the volunteer information (address and hop distance). In addition, if $TTL_a > 1$, the client retransmits the message after decreasing TTL_a by 1 and increasing hop count by 1. After a node becomes a volunteer, the volunteer initializes TTL_a as TTL_{max_a} , a system parameter to limit the coverage area of all volunteers to avoid central bottleneck. After that, the volunteer adjusts TTL_a value as maximum distance (hop count) to its clients by monitoring registration messages from the clients to minimize announcement overhead. Lifetime field indicates the valid period of the announcement. Each volunteer needs to send the announcement message again before the lifetime expires. Thus, volunteers can adjust announcement frequency as network conditions change with the lifetime field.

3.2. Service registration and discovery

Each client chooses the closest k volunteers as its local volunteers from its cached volunteer information (address and hop count). A registration message consists of hop count (the distance) to the volunteer and service description. When a client registers services with one of its local volunteers, the client lets the new volunteer know about the other $k-1$ local volunteers. This registration process makes volunteers to know other volunteers and to form a logical overlay network. Whenever a client cannot hear an announcement message from a local volunteer before the lifetime of the volunteer expires, the client assumes that the local volunteer is missing and the client needs to reregister with another volunteer. (We assume that announcement messages cannot be lost because they are broadcast messages.) If the client has other volunteers' information in its cache, the client tries to register with the nearest volunteer. Otherwise, the client needs to solicit information of volunteers in the vicinity.

Clients register with only functional information of the services. Hence, volunteers do not need to refresh transient attributes of each service, e.g. available resource or current workload, periodically. When required, such variable information is sent to a SR directly from a SP. Moreover, only when there are some changes on service functionalities, SPs update the service registry. This approach (operation is delayed until it is really needed) is flexible in dynamic situations and reduces overhead by avoiding periodic registration. With periodic service registration, update scalability can be a serious concern though it supports automatic de-registration. In VSD, de-registration is

also delayed until a volunteer realizes a SP failure (described later).

SRs adjust TTL_r value of the request message to limit reach boundary of the request. TTL_r is somewhat different from other TTL values such that TTL_r is not measured by hop count between nodes but measured by the number of volunteers to visit.

When a SR needs a service, it sends a service request message to its k local volunteers. When a volunteer receives the request message, it looks up the requested service in its directory. If the volunteer finds the matched service, it forwards the request message to the SP that provides the service instead of replying to the SR directly. Additionally, the volunteer reduces TTL_r by 1 and forwards the request message to neighbor volunteers if $TTL_r > 0$. To avoid storming responses to a SR, each volunteer may forward the service request to limited number of SPs. For load balancing among SPs, volunteers may select SPs in a round-robin fashion.) Service discovery is performed incrementally such that near services are discovered earlier and far services are discovered later.

When a SP receives a forwarded service request message from its local volunteer, it decides whether to accept the service request according to its current conditions such as workload. If a SP receives duplicate request, the SP discards it (a service request is identified by source id and message id). Otherwise, the SP may either ignore the service request by not responding, or accept it by replying to the original SR along with the detailed service specifications including service attributes, QoS related information such as available resources and current load.

A SR waits until it gets a satisfactory result before selecting a SP. The SR can select a server that provides the best preferable service among several replies by comparing details of QoS information. In addition, the SR can negotiate with several SPs before they select one SP. The SR also can save other alternative services' information for swift adaptation if the SP leaves in the middle of an ongoing service.

Delayed service de-registration may be performed at the volunteer. When a SP receives a forwarded service request message from its local volunteer, the SP needs to send acknowledgement. If the volunteer does not get acknowledgement within the given number of trials, it assumes that the SP has left and releases the SP's services registry in its directory - de-registration occurs when the SP is requested not when the SP becomes unavailable. We call this *lazy de-registration*.

3.3. Node state transition diagram

Now let us investigate VSD architecture in more detail with the node state transition diagram of Fig.1.

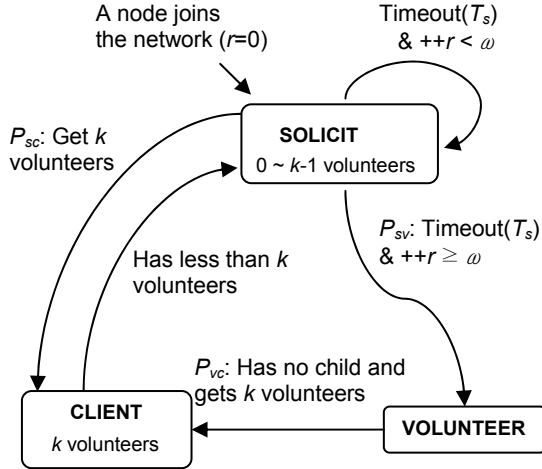


Fig. 1. Node state transition diagram

When a node N_i turns on or joins the network, N_i enters the *SOLICIT* state. N_i solicits volunteer information to its neighbors by sending an one_hop solicit broadcast message. When a neighbor node N_j receives the solicit message, N_j sends information (address and hop count) of its local volunteers to N_i . After N_i receives replies from its neighbors, N_i caches the volunteers' information and tries to register with nearest k volunteers. If a volunteer accepts N_i 's registration, the volunteer increases TTL_a when its announcement does not cover N_i . There are two cases that a volunteer cannot accept new registration: i) when the number of clients exceeds the predefined value (n_{max_c}); and ii) when $TTL_a > TTL_{max_a}$. If the volunteer cannot accept any more clients due to one of the above cases, the volunteer sends a reject message to N_i . If N_i receives the reject message from the volunteer, N_i tries again with the next nearest volunteer until it gets k local volunteers.

If N_i cannot get any reply of the *solicit* message from its neighbors or it fails to register with any k volunteers within a given time period (T_s), it repeats the above solicit process up to the given retrial time ω . If N_i cannot get k volunteers until performing solicit procedure ω times, N_i becomes a volunteer with probability P_{sv} . N_i releases all its local volunteers, if any and starts to send announcement messages. When a volunteer loses all clients due to, for example sudden movement beyond its coverage and it can get k volunteers, it enters the *CLIENT* state with probability P_{vc} .

On the other hand, if N_i gets k volunteers before completion of ω solicit procedure, it enters the *CLIENT* state from the *SOLICIT* state with probability P_{sc} . After that, if N_i loses one of its local volunteers and cannot

register with another volunteer, N_i enters the *SOLICIT* state again.

When a node is in *SOLICIT* state with no local volunteers and needs a service urgently, the node can broadcast a service request to its neighborhood, optionally.

Each node sets its own ω value by considering i) its willingness value ii) degree of mobility and iii) amount of resources. That is, ω is a function of the three state variables of the node. Different values of ω for each node make each node operate asynchronously and relatively. For example, a node strongly wants to be a volunteer and it has abundant resource and a low degree of mobility, then the node may have a low value for ω and a higher chance to be a volunteer. However, nodes can have $\omega = \infty$ if they cannot serve as volunteers.

4. Discussion on VSD

4.1. Salient features of VSD

There are several novel features of VSD. First of all, besides directory services, volunteers can also perform other network functions, such as backbone core for routing or authentication entity.

Also VSD supports selective and proactive service advertisement as follows. SPs advertise their services proactively to potential SRs that may be interested in the services. SPs can locate potential SRs using profiles if available in a manner similar to that during service discovery. For example, if a printer has a camera in the customer profile, the printer can locate all cameras in the vicinity and advertise its services to them because all cameras are potential customers. With this proactive advertisement, the SP can increase its service utilization and SRs can use the service immediately when it is needed by caching the advertisement. However, frequent service advertisements cause network and system overhead, thus volunteers need to control the amount of proactive advertisements and to be fair among SPs. In addition, service directory may maintain *advertise_allow* flag on each client entry to indicate permission to advertise. If a client does not want to be distracted with proactive advertisement, the client can request its local volunteers to set the flag '0'.

VSD is scalable in terms of request load and network size because it does not incur broadcast messages per request and each SR only consults with its local volunteers. Also VSD is robust because the system maintains k duplicate service information for each SP. Failure of a volunteer does not affect correctness of system operation even though it may downgrade performance. Moreover, adaptability to topology change can be slow or delayed due to

reliability through redundancy. VSD adapts to topology changes by letting clients dynamically replace their local volunteers. A client replaces its local volunteer not when the client can hear from a nearer volunteer than the local volunteer, but only when the client cannot hear the local volunteer's announcement. Thus the reassignment of a local volunteer is delayed as long as possible to suppress unnecessary overhead.

VSD takes physical locality into consideration to make clusters. Finding physically closer SPs without employing any location system is feasible. In addition, system performance improves because of physical proximity between a volunteer and its clients.

4.2. Optimal system parameters

To investigate system parameters of VSD for optimal performance, we analyze operation cost of VSD. We define following variables with reference to the node state transition diagram of Fig. 1. We name a node in *VOLUNTEER* state *a volunteer*, a node in *CLIENT* state *a client*, and a node in *SOLICIT* state *a solicit node*.

- n : average number of nodes in unit time ($= v+c+s$)
- v : average number of volunteers in unit time
- c : average number of clients in unit time
- s : average number of solicit nodes in unit time
- r : average number of SRs
- F_a : average announcement frequency
- λ_r : mean service request arrival rate at SRs
- λ_{cv} : mean rate for a client to change its local volunteer
- λ_{cf} : mean rate for a client to change its service functionality

We define three costs: i) costs related to volunteers; ii) costs related to client registration; and iii) costs for service discovery. We define basic average costs as follows.

- C_v : average cost for a solicit node to be a volunteer including cost for ω number of solicit procedures
- C_a : average cost per announcement
- C_{sc} : average cost for a solicit node to be a client including cost for solicit and register procedures
- C_r : average cost for a client to register or reregister its service with one of its local volunteers
- C_m : cost for a client to change one of its local volunteers including cost for procedures to solicit (option), to register with the new local volunteer and to release with the old one
- C_q : average cost per service discovery including cost for request and reply procedure

All following costs we draw are total costs in unit time.

- i) costs related to volunteers
 - a. cost for solicit nodes to become volunteers $= s P_{sv} C_v$ (1)
 - b. cost for announcements $= v F_a C_a$ (2)
- ii) costs related to client registration
 - a. cost for solicit nodes to be clients $= s P_{sc} C_{sc}$ (3)
 - b. cost for volunteers to be clients $= v P_{vc} (k C_r)$ (4)
 - c. cost for clients to reregister due to change of service functionalities $= c \lambda_{cf} (k C_r)$ (5)
 - d. cost for clients to reregister due to change of its local volunteer $= c \lambda_{cv} C_m$ (6)

Therefore the total cost related to client registration (C_{tr}) is sum of equations (3) to (6)

$$C_{tr} = s P_{sc} C_{sc} + k C_r (v P_{vc} + c \lambda_{cf}) + c \lambda_{cv} C_m$$
 (7)
 - iii) cost for service discovery $= r \lambda_r C_q$ (8)

Now, let us discuss main variables that affect the cost and should be properly set for optimal performance.

- F_a : volunteer announcement is one of main factors to cause overhead to maintain VSD architecture and F_a mainly affects the announcement cost (2). F_a need to be set high with $k = 1$ or high mobility of the volunteer. When $k > 1$, F_a can be low due to redundancy of service information. Additionally, mobility of the volunteer is low, F_a can be low with low volunteer failure.
- TTL_{max_a} : TTL_{max_a} is inverse proportional to v . Intuitively, the fewer volunteers in the system, the less is communication overhead among volunteers. On the other hand, if v increases, we can expect that cost per discovery (C_q in (8)) increases because more volunteers need to be involved to get same discovery results. In other words, high TTL_{max_a} decreases v , enlarging discovery coverage with the same TTL_r . Consequently, high TTL_{max_a} is preferable but it should be decided considering maximum coverage of a volunteer to avoid central bottleneck.
- TTL_r : TTL_r should be set according to the SR's experiments and expected density of the requested service estimated by the service's popularity.
- k : if we increase k , reliability increases to allow a lower value of F_a . However, a high value of k increases total cost for client registration (7) and workload per volunteer due to increment of average cluster size, i.e. average cardinality of the volunteer region ($= c k / v + I$).

4.3. Connectivity among volunteers

The value of k is one of the main factors that impact system performance. Intuitively and due to similar case

study by Yang et al. [19], we choose $k = 2$ and investigate connectivity among volunteers in this subsection.

A volunteer maintains neighbor volunteers' addresses through registration messages of clients as shown in Fig. 2. Let us say client g has two local volunteers A and B . When g registers with A , g informs A about B . Similarly, when g registers with B , g lets B know about A . Thus A and B maintain each other's address.

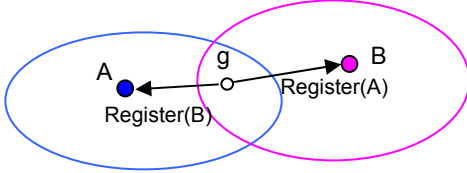
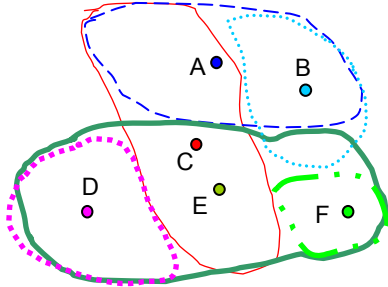


Fig. 2 A and B know each other through a client g

Now, let us investigate the effect of connectivity among volunteers when $k = 2$. To make things simple, we assume that every node has the same transmission radius. We may represent a target network as an undirected graph $G(V, E)$, where V is a finite set of nodes including volunteers and clients and E is a set of neighbor relationship between two nodes. Also we assume that G is connected, which means that every node is reachable from every other. When $k = 2$, every client belongs to exactly two volunteers' regions. We denote a region of volunteer A as R_A . In Fig. 3, we illustrate an example of G . In this figure, we just draw volunteers and boundaries of their regions and we omit clients and links between nodes. Therefore overlapped clusters are formed as in Fig. 3.



Volunteer	Neighbors' info.	Region
A	B, C	---
B	A, E	---
C	A, E	---
D	E	---
E	B, C, D, F	---
F	E	---

Fig. 3 Regions in graph G with $k = 2$

For volunteers A and B , if $R_A \cap R_B \neq \emptyset$, we say that there is a virtual connectivity between A and B and we denote as $A \leftrightarrow B$.

Now, we define a logical undirected graph $G_L(V_V, E_C)$, where V_V is a set of volunteers in G and E_C is a set of virtual connectivity relationship between two volunteers as in Fig. 4.

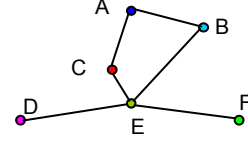


Fig. 4 A logical graph $G_L(V_V, E_C)$ of G in Fig. 3.

Virtual connectivity among volunteers is important for successful service discovery by expanding coverage area of discovery with TTL_r . If there is no virtual connectivity among volunteers, the service request can not be forwarded to further volunteers even with high value of TTL_r . Then, how can we improve to provide virtual connectivity among volunteers? There are following choices.

1. increase k
2. when a client registers with one of its local volunteers, the client shares information about all other volunteers in its cache rather than only the other local volunteers.
3. along with choice 2, increase TTL_a of all volunteers by 1.
4. a volunteer collects neighbor volunteers' addresses through other volunteers' announcements in addition to common clients' registration.

Choice 3 guarantees perfect connectivity, but choices 1 and 3 incur high overhead as shown in Subsection 4.2. Therefore choices 2 and 4 are preferable. Especially they are very helpful to improve connectivity when $k = 1$.

5. Simulation Studies

5.1. Simulation setup

We performed simulation studies using NS2 version 2.27 [20]. There are two objectives of this study: i) to identify the proper values of system parameters for optimal performance and to validate the hypothesis of Subsection 4.2; and ii) to compare with the pull scheme and to demonstrate the superiority of the VSD scheme in MANETs (Mobile Ad hoc NETWORKs).

We choose MANETs for our simulation scenarios. As we mentioned, VSD is targeting pervasive computing environments subject to heterogeneity and

uncertainty. More capable and stable nodes carry out directory services as volunteers. However, in our experiments, every node has same capability and degree of mobility forming the worst scenarios for VSD. Our premise is that, if it can be shown that VSD achieves comparable performance in the above scenarios, we can infer VSD performs well in normal pervasive computing environments. We introduce uncertainty by increasing maximum speed of nodes and simulating abrupt node departures. In addition, we choose the well known broadcast based pull scheme for comparison with VSD. The pull scheme operates such that whenever a SR needs a service, it floods a request message with TTL_r and any SP having a matched service replies to the SR.

Total simulation time is 700 seconds and we trigger service requests for 10 minutes. Default number of SRs is 20 and default rate of service request per SR is 3 per minute. Therefore, total 600 service requests occur during simulation. Each experiment is run 10 times and the average values are considered. Distributed coordination function (DCF) of IEEE 802.11 is used for MAC protocol. The radio interface is based on Lucent's WaveLan technology with 250 meters of nominal propagation range and 2Mbps of nominal bit rate. Radios use omni-directional antennas and we assume a two-ray ground propagation model. We use random waypoint mobility model [21]. We summarize other simulation parameters with default values in Table 1. (Note that parameters in all experiments in this section use these default values unless specified)

parameters	Values (default value)		
Number of nodes (n)	50	(100)	200
Network area (m^2)	700 ²	(1000 ²)	1500 ²
number of service types (n_s)	20	(40)	80
TTL_{max_a}	(3)	1, 2, 3, 4, (5)	
TTL_r	1, (2), 3		
F_a (per minute)	0.5, (1), 2		
Maximum node speed (m/s)	(5), 10, 20		
Pause time (seconds)	(5)		
Routing protocol	(DSDV), AODV		
k	1, (2), 3		

Table 1. Simulation parameters

Without considering node heterogeneity in VSD, we assign ω uniformly and randomly from 1 to 20 to each node when the node starts up. Another important parameter is the number of service types (n_s). Every node is a SP that provides a service of a certain type. A service type is assigned to each node in a round robin fashion. If there are 10 service types in a network with 50 nodes, 5 SPs provide a same service. Therefore, with high n_s , service density ($d_s = n / n_s$, average number of SPs providing a same type of service) is low and vice versa.

The main outputs from simulation study are:

- n_t : total number of messages sent during simulation
- d_r : average delay of the first reply on successful requests
- n_r : average number of replies per successful request within a given duration (set as 1 seconds)
- r_f : failure rate of service discovery on total requests

In addition, we define utility value δ as a function of main observations as follows,

$$\delta = f(r_s, r_d, u_r) = r_s \times r_d / u_r \text{ where,}$$

- r_s (success rate) = $1 - r_f$
- r_d (discovery rate) = n_r / d_s
- u_r (resource usage) = $n_t / \text{number of total requests}$

To observe robustness of the system, we simulate two node leaving scenarios with 30% and 70% nodes leaving. With these scenarios, performance degradation may be due to volunteer failure or SP failure.

5.2. Optimal system parameters in VSD

In this subsection, we mainly try to figure out the best values for system parameters such as k , TTL_{max_a} , TTL_r and F_a in VSD. Fig. 5 shows failure rates when $n = 50$ and 100 with various maximum node speeds (S_{max}) and various k values.

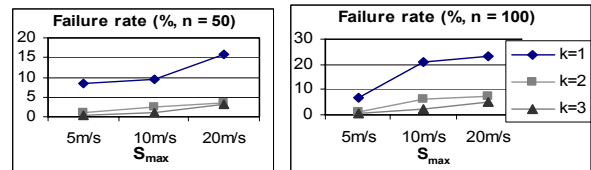


Fig. 5 Failure rate (%) with $n = 50$ and 100

With $k = 1$, if the local volunteer of a SR cannot find the matched service, the request fails even with $TTL_r = 2$ because mostly volunteer connectivity is not provided. (We implemented only the fourth choice in Subsection 4.3 to improve connectivity.) In addition, with $k = 1$, we observe that reply delays are longer and variance of results including failure rate is much higher than those with $k = 2$ or 3. However, failure rates when $k = 2$ are a little bit higher than when $k = 3$.

We conducted experiments for two different (30% and 70%) nodes leaving scenarios. Fig. 6 shows failure rates when $n = 100$.

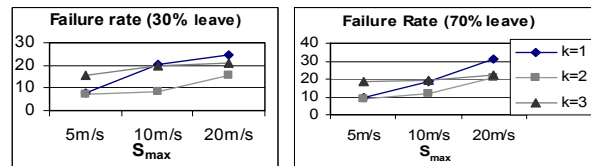


Fig. 6 Failure rate (%) for node leaving scenarios ($n=100$)

We observe that, failure rates with 50 nodes have similar trends to those without node leaving scenarios. That is, results with $k = 3$ show slightly lower failure rate than those with $k = 2$ and failure rates with $k = 1$ are the highest. For $n = 100$, the failure rates is lower for $k = 2$ than those for $k = 3$. With $k = 3$, much more traffic is incurred in very dynamic situations because many clients need to reregister with new local volunteers. Moreover, when $TTL_r = 2$ and $k = 3$, high proportion of requests are forwarded to same volunteers, thereby incurring high message collision. We observe that for $k = 3$, n_t is much higher and utility values are a lot lower than corresponding values for $k = 2$ and this situation is getting worse as n increases. Therefore, we recommend $k = 2$.

We also ran experiments with various TTL_{max_a} , when $TTL_r = 1$. We do not limit the maximum number of clients per volunteer (n_{max_c}), thus only TTL_{max_a} affects actual TTL_a . Therefore with a higher value of TTL_{max_a} , volunteers mostly have a higher TTL_a . As we expected in the cost evaluation, with large TTL_{max_a} , the results are better in terms of failure rate. However, utility values are almost same with varying TTL_{max_a} because with a higher value of TTL_{max_a} , resource usage (u_r) increases. In addition, we learned that generally, experiments using AODV show better performance than those using DSDV, except for reply delay. d_r is around 0.3 ~ 0.6 seconds with AODV and around 0.2 ~ 0.4 seconds with DSDV.

In addition, we performed experiments with varying F_a (0.5 ~ 2 per minute) with $TTL_r = 1$ and $TTL_{max_a} = 2$. Again, as we expected, failure rate is decreasing as F_a is increasing especially in scenarios with high node speed. Additionally, utility value is decreasing as F_a is increasing due to increase of n_t . We infer that low values such as once per hour may be enough for F_a when volunteers are almost static, which is ideal and expected situation for VSD. Consequently, overhead of announcement can be reduced significantly.

Fig. 7 presents failure rates and n_t against varying TTL_r between 1 ~ 3 when $TTL_{max_a} = 1$ and the number of SRs = 5.

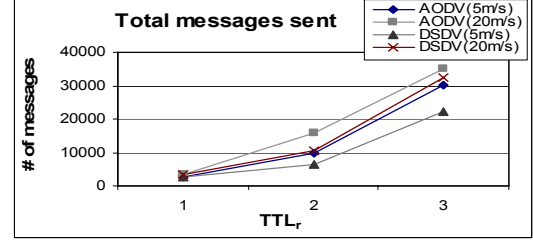
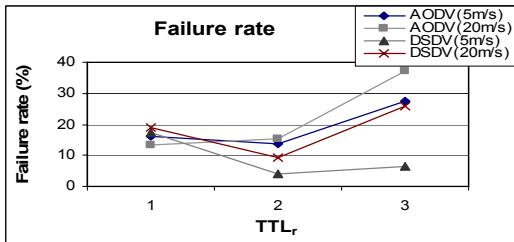


Fig. 7 Failure rate and n_t ($TTL_{max_a} = 1$)

In Fig. 7, we see that in most cases failure rates with $TTL_r = 2$ are lower than those with $TTL_r = 3$ and $TTL_r = 1$. Intuitively, we think that a greater value of TTL_r decreases failure rate. However, with $TTL_r = 3$, requests are forwarded to many volunteers repetitively. Thus message collision happens often at volunteers and at SPs especially with high service density (d_s). We confirm this with very high n_t when $TTL_r = 3$. Besides, due to high collision, DSDV shows better performance than AODV when $TTL_r \geq 2$. These experiments suggest avoiding higher values of TTL_r .

We also performed experiments of most scenarios with $n = 200$ and 50. However, the results show similar trends, thus we do not mention those here.

5.3. Comparison with the pull scheme

In this subsection, we compare VSD with the pull scheme in terms of scalability, discovery rate and robustness.

Fig. 8 shows n_t against various numbers of SRs to investigate scalability. With the pull model, we use 2 and 5 for TTL_r . (Note that TTL_r in the pull model is operated by hop count between nodes)

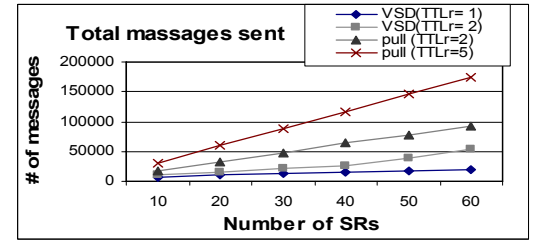


Fig. 8 n_t ($S_{max} = 5$ m/s)

Compared to the pull model, VSD reduces significant communication overhead in terms of n_t , thus utility values are much higher. The failure rates are 3 ~ 7 % in VSD scenarios using $TTL_r = 1$ and 1 ~ 3 % using $TTL_r = 2$. In the pull scenarios, failure rates are a bit higher than those with VSD. When $TTL_r = 2$, failure rates are 9 ~ 12 % and when $TTL_r = 5$, they are 4 ~ 8 %. However, r_d is very low with the pull scheme which is around 0.06 ~ 0.2 seconds, but with VSD, r_d is around 0.1 ~ 0.4 seconds.

To compare probability of successful discovery, we carried out experiments with various numbers of service types (n_s) when $S_{max} = 5\text{m/s}$ and 20m/s . Fig. 9 shows failure rate and utility value when $S_{max} = 20\text{m/s}$. We use $TTL_r = 1$ in VSD and $TTL_r = 2$ in the pull scheme when $n_s = 10$ and 20 . Similarly, we use $TTL_r = 2$ in VSD and $TTL_r = 5$ in the pull scheme when $n_s = 40$ and 100 .

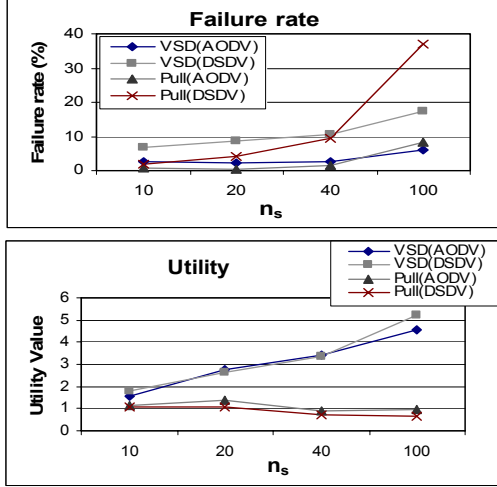


Fig. 9 failure rate and utility ($S_{max} = 20\text{ m/s}$)

In both scenarios with $S_{max} = 5\text{ m/s}$ and 20 m/s , output trends are similar, but when $S_{max} = 20\text{ m/s}$, most performance is worse than when $S_{max} = 5\text{ m/s}$. As n_s increases, which means services become scarce, VSD presents much better performance in terms of failure rate, bandwidth usage, and utility value. In pervasive computing environments, we expect various kinds of services exist, thus discovery capability for scarce services is greatly appreciated. We observe that with VSD, n_t does not much increase as n_s increases (TTL_r also increases in this experiments) because n_r is reduced. However, with the pull scheme, n_t increases suddenly with $TTL_r = 5$ when $n_s = 40$ and 100 , thus utility value decreases. Different from results when $S_{max} = 5\text{ m/s}$, when $S_{max} = 20\text{ m/s}$, failure rates with VSD are higher than those with the pull scheme when number of service types ≤ 40 . However, utility values with VSD are much higher than those with the pull scheme.

We also performed experiments to compare node leaving scenarios for both schemes. We observe that VSD is very robust even when quite high percentages of nodes are leaving during simulation in term of successful service discovery capability. n_t and utility value show similar trend in both scenarios with $S_{max} = 5\text{ m/s}$ and 20 m/s . With VSD, even though total number of nodes in the system is reduced due to nodes' departure, n_t is almost same because other messages (for example, reregistration due to loss of local

volunteers) are incurred with high network dynamism. On the other hand, in the pull scheme, n_t is directly dependent on the number of nodes in the system. However, even in 70 % nodes leaving scenarios, VSD shows better performance in terms of utility value and bandwidth usage. With $S_{max} = 20\text{ m/s}$, failure rates are a little bit worse with VSD than those with the pull scheme using $TTL_r = 5$, but n_t for the full scheme is almost 3 ~ 4 times higher than those for VSD.

To summarize, in high dynamic situations where all nodes move rapidly, the pull scheme with high TTL_r shows slightly lower failure rate than VSD. However, the pull scheme requires too much network usages (low utility values) and does not scale well. To the contrary, VSD supports high scalability and high discovery rate of scarce services. Moreover, in most cases, VSD shows high utility values and lower failure rates. Through experiments using the worst MANET scenarios (for VSD) where every node is homogeneous (have same node speed), we validate that VSD is practical and preferable choice for normal pervasive computing environments.

6. Conclusion and Future Work

We propose VSD – a flexible and adaptable protocol for service discovery targeting pervasive computing environments. VSD exploits node heterogeneity in terms of mobility and capability to allow more stable and capable nodes to provide directory services voluntarily. VSD adapts to topology change easily and readjusts automatically, thus it supports self-reconfiguration. Overlapped clustering improves reliability through redundancy and improves scalability through distribution. VSD also supports locality by network proximity to retrieve closer service providers.

We present performance of VSD through comprehensive simulation studies and compare with the traditional pull scheme. By showing that VSD outperforms the pull scheme in most MANET scenarios, we confirm that VSD is a flexible and adaptable architecture appropriate for dynamic pervasive computing environments.

There are several future works we plan to perform. First of all, we will consider exploiting other network functions using VSD architecture. Making use of volunteers for other purposes can compensate overhead to maintain the system architecture by increasing utilization of volunteers. Second, we need to employ security mechanism in VSD to determine trustworthiness of volunteers, service providers and service requestors. This role is critical in an open environment, where there are many opportunities for misuse, both from fraudulent volunteers and misbehaving clients. Again, for security support, we

may make use of volunteers as authentication entities. Third, we will expand our simulation study including comparing other service discovery schemes such as the push model and hybrid model of push and pull. Additionally, we will employ actual node mobility to simulate node heterogeneity for common pervasive computing environments.

7. Acknowledgements

This material is based on work supported by the National Science Foundation under Grant No. 0129682.

8. Reference

- [1] M. Weiser, "The Computer for the 21st Century," *Scientific American*, Sept., 1991, reprinted in *IEEE Pervasive Computing*, Jan.-Mar. 2002, pp. 19-25.
- [2] D. Saha and A. Mukherjee, "Pervasive Computing: a Paradigm for the 21st Century", *IEEE Computer*, March 2003, pp 25 – 31
- [3] M. Satyanarayanan, Editor, "Dealing with uncertainty" *IEEE Pervasive computing Volume: 2, Issue: 3, Year: July-Sept. 2003*
- [4] Sun Microsystems, *Jini Architecture Specification*, June 2003
- [5] E. Guttman, C. Perkins, J. Veizades, and M. Day, "Service Location Protocol, Version 2," *IETF RFC 2608*, June 1999.
- [6] Microsoft Corporation, "Understanding Universal Plug and Play, www.upnp.org/download/UPNP_understandingUPNP.doc," A White Paper, June, 2000.
- [7] S. Helal, N. Desai, V. Verma and C. Lee, "Konark--A Service Discovery and Delivery Protocol for Ad-hoc Networks," *Proceedings of the Third IEEE Conference on WCNC, New Orleans, March 2003*.
- [8] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: Scalable Peer-to-Peer Lookup Service for Internet Applications," *IEEE/ACM Transactions on Networking*, vol. 11, No. 1, Feb. 2003.
- [9] S. Ratnasamy, P. Francis, M. Handley, R. Karp, and S. Shenker, "A Scalable Content Addressable Network," *In: Proceedings of ACM SIGCOMM*, Aug. 2001.
- [10] A. Rowstron and P. Druschel. "Pastry: Scalable, distributed object location and routing for large-scale peer-to-peer systems," *Proc. The 18th IFIP/ACM Intl. Conf. on Distributed Systems Platforms*, Nov. 2001
- [11] H. Balakrishnan, M. F. Kaashoek, D. Karger, R. Morris, and I. Stoica, "Looking Up Data in P2P Systems," vol. 46, No. 2, *Communications of the ACM*, Feb. 2003.
- [12] Ulas C. Kozat and Leandros Tassiulas, "Network Layer Support for Service Discovery in Mobile Ad Hoc Networks", *Proc. IEEE INFOCOM 2003*
- [13] D. Chakraborty, A. Joshi, T. Finin and Y. Yesha, "Towards Distributed Service Discovery in Pervasive Computing Environments", *IEEE Transactions on Mobile Computing*, July. 2004.
- [14] Jiangchuan Liu, Qian Zhang, Bo Li, Wenwu Zhu, and Jun Zhang, "A Unified Framework for Resource Discovery and QoS-Aware Provider Selection in Ad Hoc Networks," *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 6, Issue 1, January 2002.
- [15] S. Raghupathy, S. Prasun, and B. Vaduvur, "CEDAR: A core-extraction distributed ad hoc routing algorithm," *IEEE Journal on Selected Areas in Communications*, vol. 17, no. 8, pp. 1454–1465, Aug. 1999.
- [16] C.R. Lin and M. Gerla, "Adaptive Clustering for mobile wireless networks," *IEEE Journal on Selected Areas in Communications*, vol. 15, no. 7, Sep. 1997.
- [17] W. R. Heinzelman, A. Chandrakasan, H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks," *Proc. HICSS-33*, Jan. 2000.
- [18] M. Klein, B. Konig-Ries, "Multi-Layer Clusters in Ad-hoc Networks – An Approach to Service Discovery," *In: Proceedings of the First International Workshop on Peer-to-Peer Computing*, Pisa, Italy. (2002) 187–201
- [19] B. Yang and H. Garcia-Molina, "Designing a Super-Peer Network," *In Proc. of the 19th International Conference on Data Engineering*, March 2003.
- [20] K. Fall and K. Varadhan, *The ns manual (formally ns notes and documentation)*. <http://www.isi.edu/nanam/ns/doc/index.html>. Cited 13, December 2003.
- [21] J. Broch, D. A. Maltz, D. B. Johnson, Y. Hu, J. Jetcheva, "A Performance Comparison of Multi-Hop Wireless Ad Hoc Network Routing Protocols", *Proc. 4th Annual ACM/IEEE Int. Conf. on Mobile Computing and Networking*, Oct. 1998.